FPT UNIVERSITY

HO CHI MINH CITY CAMPUS

LINUX OPERATING SYSTEM

ASSIGNMENT REPORT

Linux Kernel Device Drivers: ioctl Character Device and Misc Counter Driver

Course	LOS201 - Linux Operating System
Lecturer	Phạm Thế Vinh
Student name	Trần Nguyễn Việt Dũng
Student ID	SE205180
Major	Integrated Circuit Design / Semiconductor Engineering
Date	Ho Chi Minh City, 21 October 2026

Embedded Linux concepts applied to an ARM-based development environment

EXECUTIVE SUMMARY

This report studies two Linux kernel modules in an embedded ARM environment. The first module, asgn1_driver, is a character driver that exposes a small byte buffer and management functions through an ioctl interface. The second module, counter_driver, uses the Linux miscellaneous-device framework to provide a software counter through /dev/counter and status information through procfs.

The work focuses on the boundary between user space and kernel space: file operations, ioctl command encoding, safe data transfer, shared-state synchronization, device registration, and test-driven verification. The target environment described by the reference assignment is QEMU emulating an ARMv7 vexpress-a9 platform with a Cortex-A9 CPU and an initramfs.

OBJECTIVES

- Explain how an application reaches a kernel driver through the VFS and system calls.
- Describe a character driver with an ioctl control interface and a mutex-protected state.
- Describe a misc-device counter that uses atomic operations and correct read-offset handling.
- Compare character, block, and network device models and explain key safety decisions.
- Define a repeatable validation plan using shell scripts, procfs output, and dmesg logs.

REPORT OUTLINE

Section 1 presents the system architecture and interfaces. Sections 2 and 3 describe the two drivers, their validation plans, and the corresponding terminal evidence. Section 4 compares driver models and synchronization choices, Section 5 answers kernel-programming theory questions, and Section 6 maps the included evidence to the behaviors it demonstrates.

REPRODUCIBILITY NOTE

The terminal captures included after the validation plans record the observed results for this run. Task 1 ends with 4 PASS and 0 FAIL; Task 2 ends with 7 PASS and 0 FAIL. Retain the source code and raw terminal logs with these screenshots to support reproducibility.

1. SYSTEM ARCHITECTURE AND PROJECT STRUCTURE

The design separates user-space test tools, Linux entry points, device nodes, driver logic, and the emulated execution platform. A user program invokes open(), read(), write(), or ioctl(). The Virtual File System (VFS) dispatches each operation to the file_operations registered by the matching driver. Device nodes under /dev expose device functionality, while procfs entries provide human-readable state and statistics.

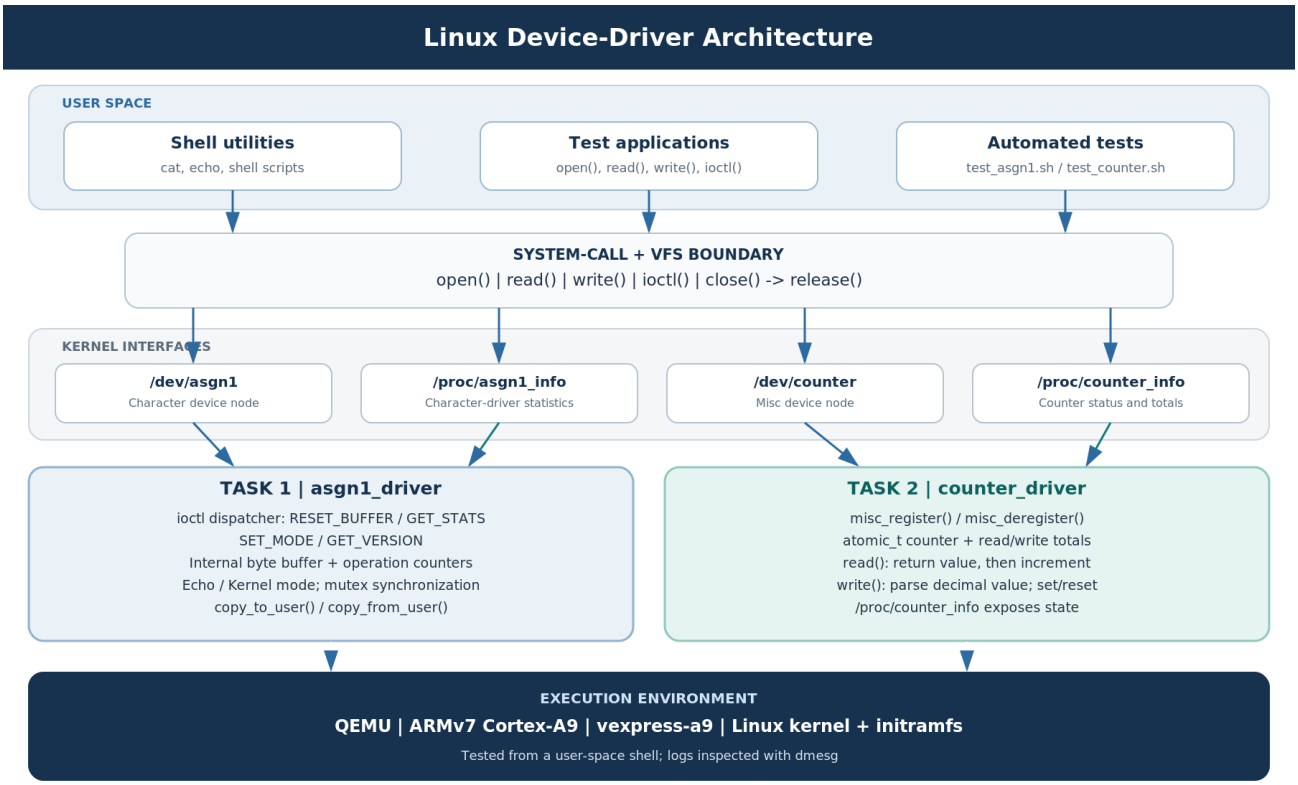


Figure 1. High-level architecture and user-to-kernel data flow.

1.1. Project structure

A clear directory layout keeps source code, test utilities, captured evidence, and the report separate. A practical layout for this assignment is:

```
Assignment_1/
 task1_asgn1_driver/
  asgn1_driver.c
  asgn1_ioctl.h
  test_asgn1.sh
 task2_counter_driver/
  counter_driver.c
  test_counter.sh
 logs/
  task1_test_output.txt
  task2_test_output.txt
  dmesg_output.txt
 report/
  Linux_Device_Driver_Report.docx
  architecture_diagram.png
```

For an IC-design student, this software layer is relevant because embedded SoCs need kernel drivers to connect applications with platform peripherals, memory-mapped registers, and on-chip controllers. QEMU allows early driver-interface checks without requiring a physical development board.

1.2. DRIVER INTERFACE SPECIFICATIONS

The two modules expose different registration models and synchronization mechanisms. The distinction is intentional: Task 1 exercises an explicitly assigned character-device major number and ioctl dispatch, while Task 2 demonstrates the simpler misc-device framework.

Aspect	asgn1_driver (Task 1)	counter_driver (Task 2)
Registration	register_chrdev(); fixed major chosen in the permitted range; node created with mknod in the test setup.	struct miscdevice with misc_register() / misc_deregister(); shared major 10 (MISC_MAJOR).
Device node	/dev/asgn1	/dev/counter
Interfaces	open(), read(), write(), release(), unlocked_ioctl(); /proc/asgn1_info.	open(), read(), write(), release(); /proc/counter_info.
Driver state	Internal buffer, open/write/read counters, buffer length, last write size, and mode.	Atomic counter, total successful reads/increments, and total writes/sets.
Synchronization	Mutex protects the shared buffer and related counters.	atomic_t operations for the counter; statistics must follow a consistent update policy.
User/kernel transfer	copy_from_user() and copy_to_user(); reject invalid pointers with an error.	Copy a bounded input string, parse with kstrtoint(), and return a proper error for invalid input.
Validation tool	test_asgn1.sh plus a small ioctl test application.	test_counter.sh with reads, writes, reset, and procfs inspection.

1.3. ioctl command specification

The ioctl interface adds control operations that do not fit naturally into a byte-stream read or write. Command direction should reflect the perspective of the user-space caller: _IOR copies data from the kernel to user space, and _IOW copies data from user space into the kernel.

Command	Macro	Argument	Expected behavior
ASGN1_RESET_BUFFER	_IO	No argument	Clear the buffer and reset read_count and write_count; a subsequent read should return no payload.
ASGN1_GET_STATS	_IOR	struct asgn1_stats	Return open_count, write_count, read_count, buffer_len, last_write_size, and mode.
ASGN1_SET_MODE	_IOW	int	Mode 0 prepends [KERNEL]; mode 1 stores the bytes unchanged. Other values return -EINVAL.
ASGN1_GET_VERSION	_IOR	char[64]	Copy a bounded version string to user space.

The ioctl handler uses a switch over the four supported request values. An unknown request returns -ENOTTY, and the driver logs each request so that a test can correlate application activity with dmesg output.

2. CORE MODULE IMPLEMENTATION: asgn1_driver

2.1. Design and operation flow

The module is organized into three logical layers. The interface layer defines file_operations and dispatches unlocked_ioctl requests. The operation layer applies the selected mode, updates the statistics, and handles the reset or version operation. The data layer owns the byte buffer, counters, mode flag, and version string.

- Shared interface header: asgn1_ioctl.h defines the magic value, the asgn1_stats structure, and all ioctl command macros. The same header should be included by the kernel module and the user-space test to avoid mismatched request values.
- Mutual exclusion: a mutex guards reads and updates to the buffer and counters. Locks should be held only as long as required, and every return path must release a lock already acquired.
- Safe data transfer: copy_from_user() and copy_to_user() are used when exchanging application buffers or structures. These helpers can fail; a partial or invalid transfer should be reported rather than treated as success.
- Diagnostics and cleanup: open(), release(), write(), and ioctl calls emit identifiable messages. Module exit unregisters the device and removes procfs resources before the module memory is released.

2.2. Important behavior

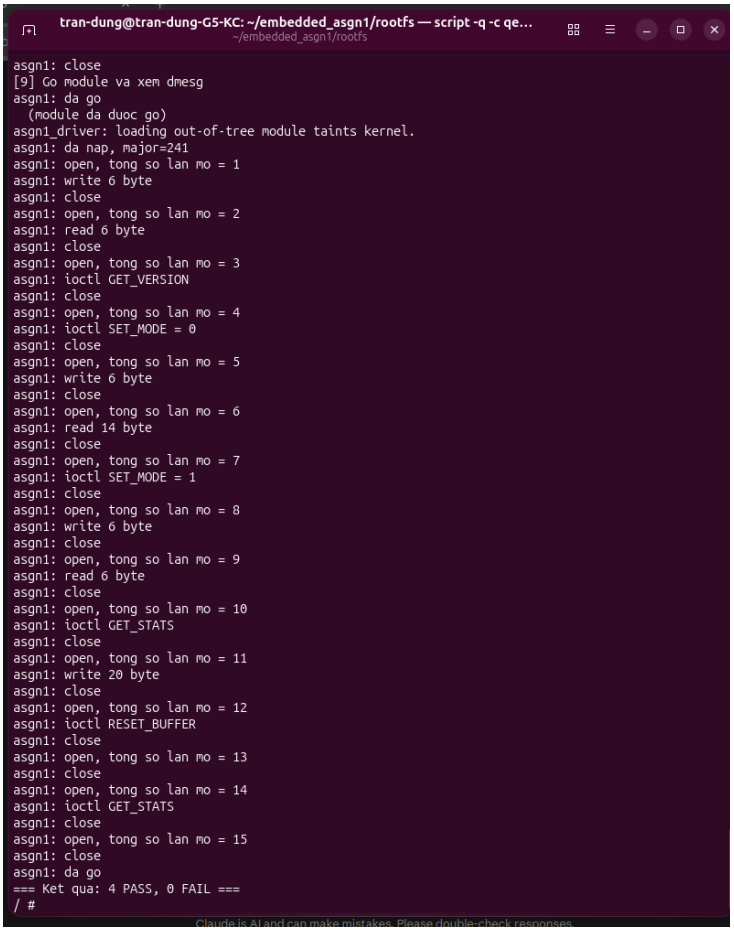
Operation	Design rule	Reason
write()	Bound the number of bytes copied and update last_write_size and write_count under the mutex.	Prevents buffer overflow and inconsistent statistics.
read()	Return no more than the available payload; update read_count according to the documented counting rule.	Supports normal file semantics and useful diagnostics.
SET_MODE	Copy the integer safely and accept only the supported values 0 and 1.	Rejects unsupported states with -EINVAL.
GET_STATS	Initialize the output structure and copy it to user space.	Avoids returning uninitialized kernel data.
Unknown ioctl	Return -ENOTTY and log the request value.	Makes unsupported operations explicit and testable.

2.3. Functional validation plan

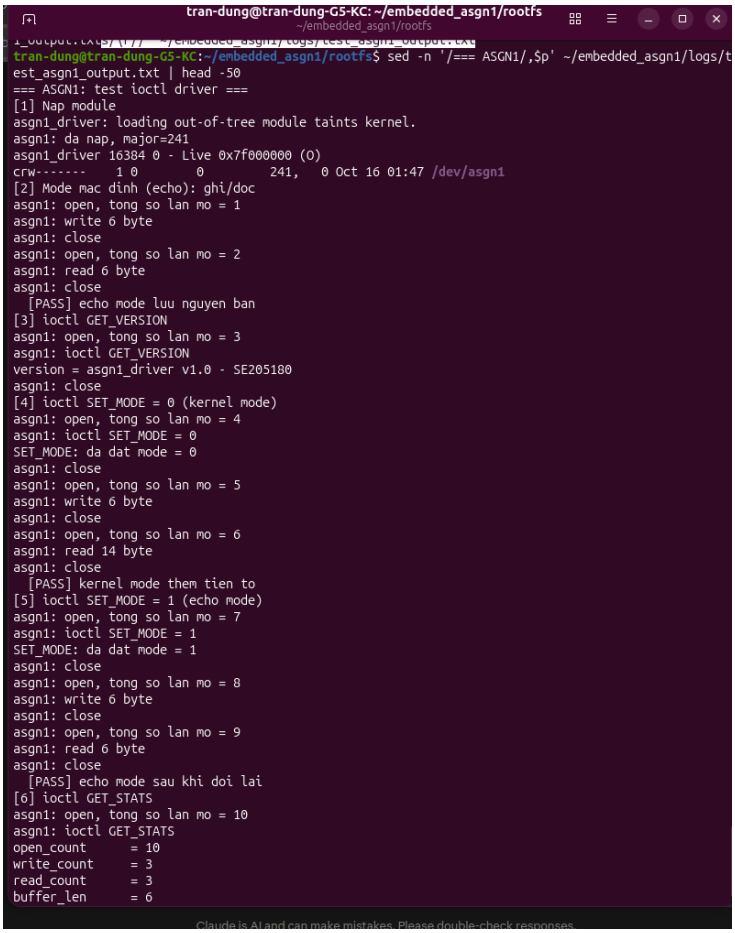
The Task 1 test should load the module, create /dev/asgn1 if required, exercise both write modes, query statistics and version, reset the state, send invalid requests, and unload the module. A test script should fail immediately when an assertion does not match the expected output.

Test case	Expected result	Validation method
Write/read in Echo mode	The output matches the written data.	Compare the bytes read from /dev/asgn1.
Write in Kernel mode	The stored output has the [KERNEL] prefix.	Inspect the returned string.
GET_STATS	All fields match the operations performed.	Compare every struct field in the ioctl test app.
GET_VERSION	Returns a bounded version string.	Check the output and buffer length.
RESET_BUFFER	Buffer length and selected counters become zero.	Run the reset ioctl and then read the node.
Invalid ioctl request	Returns ENOTTY for an unsupported request.	Check the error message and corresponding dmesg entry.

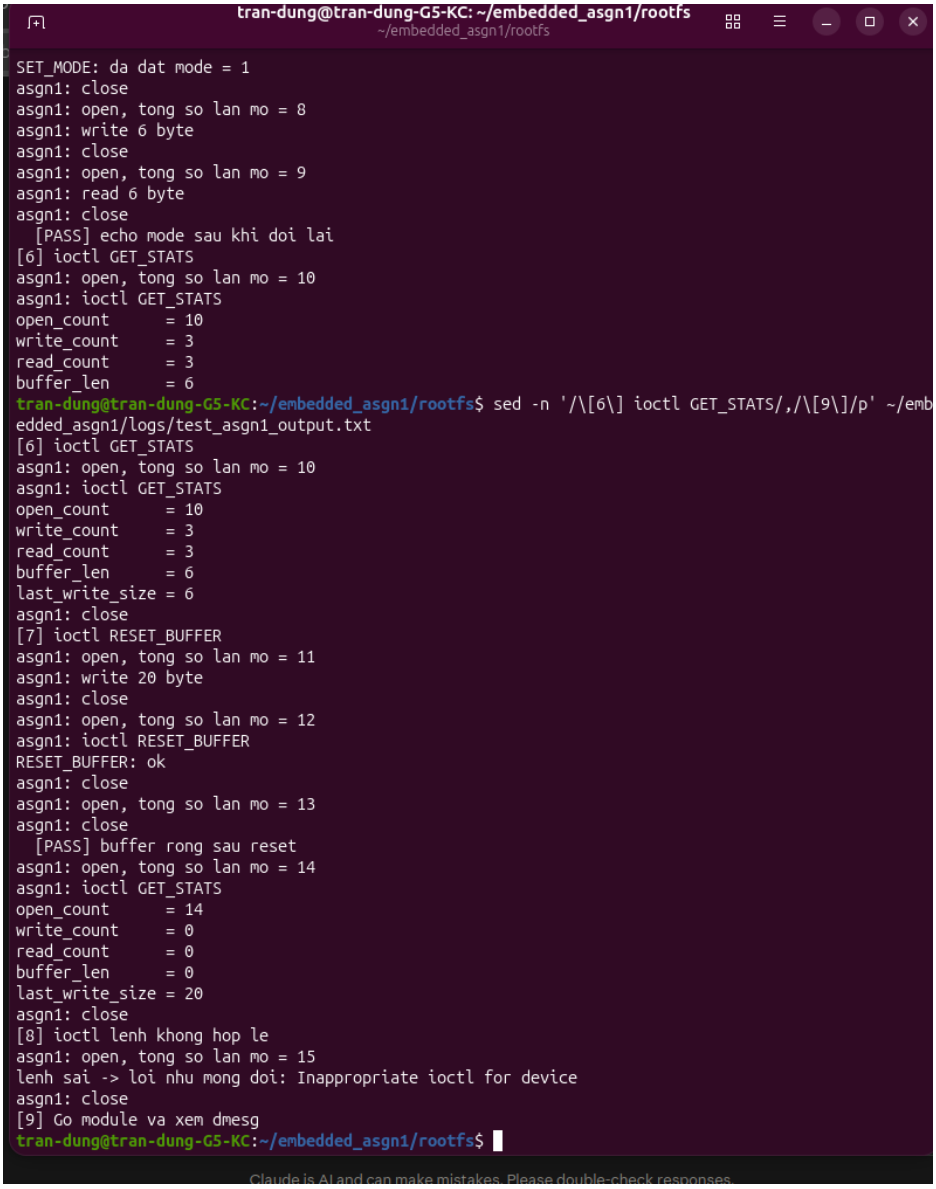
Observed result: the supplied Task 1 terminal capture ends with 4 PASS and 0 FAIL. The screenshots show driver logs, Echo and Kernel modes, GET_VERSION, GET_STATS, RESET_BUFFER, and rejection of an unknown ioctl request. Only behavior visible in the captured output is treated as demonstrated evidence.



Evidence 01. Task 1 Driver Operation Logs and Final Test Summary.



Evidence 02. Verification of Driver Modes, Version Information and Statistics.



Evidence 03. Verification of Buffer Reset and Invalid ioctl Handling.

3. MISC DEVICE DRIVER: counter_driver

3.1. Registration and data model

counter_driver models a simple software counter using the Linux misc-device framework. A static struct miscdevice describes the device name and file operations. misc_register() publishes the device when the module is loaded; misc_deregister() removes it during module exit. The resulting device node is /dev/counter when devtmpfs or udev creates it, or when the test environment supplies the node.

The main state is an atomic_t counter. The read operation returns the current value as decimal text followed by a newline and then increments the counter. The write operation copies a bounded input string, parses it with kstrtoint(), and sets the counter to the requested value. Writing 0 can be used to reset it. Negative values or non-numeric input should be rejected if the assignment contract disallows them.

The /proc/counter_info entry reports the current value and operation totals. Procfs is used for status inspection, while /dev/counter remains the primary read/write interface.

3.2. File semantics and error handling

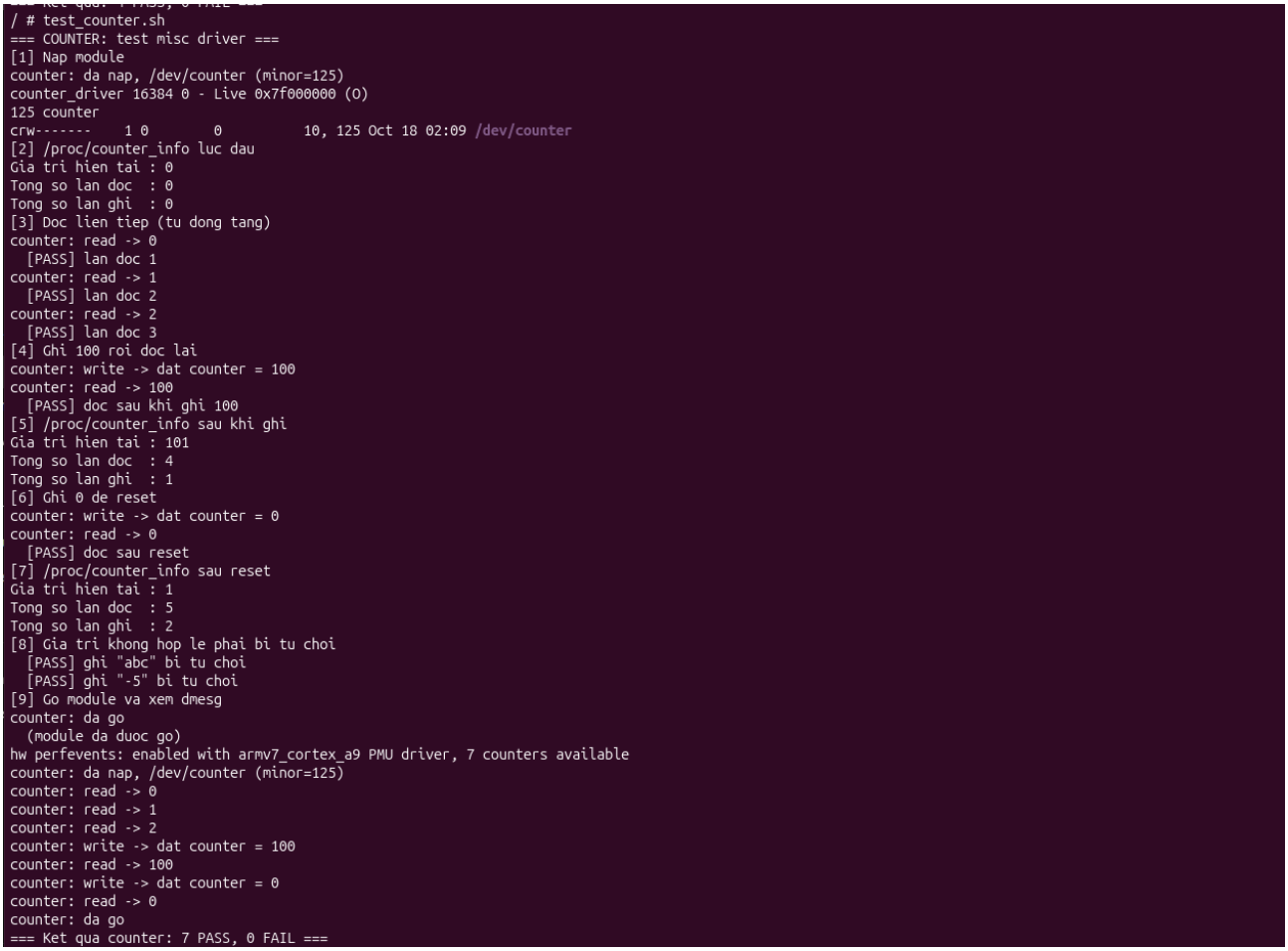
- The read handler checks *ppos. On the first successful read it copies one formatted value to user space and advances the offset; a subsequent read on the same file description returns 0 (end of file).
- The write handler terminates and parses the copied input safely. It returns the byte count only after the value has been accepted; malformed text returns -EINVAL.

- Atomic operations prevent lost updates to the counter itself. If a precise “return the old value and increment exactly once” behavior is required under concurrency, use a single atomic fetch-and-increment style operation where supported by the target kernel API, rather than a separate read followed by increment.
- The module should include appropriate MODULE_LICENSE, MODULE_AUTHOR, and MODULE_DESCRIPTION metadata and unregister its procfs entry and misc device during cleanup.

3.3. Functional validation plan

Step	Expected output or state	Pass criterion
Initial procfs check	Counter = 0; total reads = 0; total writes = 0.	All initial fields are zero.
Read sequence	Three sequential reads return 0, 1, and 2 from the initial state.	Each accepted read increments the counter and read total once.
Write 100 then read	The next value returned is 100, followed by an increment.	Write sets the requested value.
Write 0	Writing 0 resets the counter; the test then reads back 0.	The returned value is 0, and the next procfs check shows the post-read value.
Final procfs check	Current value = 1; total reads = 5; total writes = 2.	Reported values match the operations in the captured run.
Unload	No registration or cleanup error.	Module no longer appears in lsmod; dmesg has cleanup message.

Observed result: the Task 2 terminal capture ends with 7 PASS and 0 FAIL. Three sequential reads return 0, 1, and 2. Writing 100 and reading returns 100; writing 0 and reading returns 0. Because each read increments the counter after returning its value, the final /proc/counter_info output reports current value 1, total reads 5, and total writes 2. The test also rejects the non-numeric input “abc” and the negative input “-5”.



Evidence 04. Counter Driver Functional Test and Validation Results.

4. DRIVER DESIGN ANALYSIS

4.1. Character, block, and network devices

Criterion	Character device	Block device	Network device
Kernel interface	file_operations	block-device operations and request queue	net_device_ops and networking stack
Data unit	Byte stream	Block/sector, commonly 512 B or 4 KiB	Packet/frame
Access pattern	Sequential or control-oriented; may support ioctl	Random access to logical blocks	Asynchronous transmit/receive
Buffering model	Driver-owned buffer/FIFO where needed	Page/buffer cache and I/O scheduling	Packet queues, sk_buff, DMA rings, NAPI
Typical example	UART, GPIO, simple sensor, /dev/counter	eMMC, SD card, mass storage	Ethernet, Wi-Fi, CAN networking

An I2C temperature sensor that returns a small sample is commonly exposed as a character device or through an appropriate subsystem interface. eMMC storage needs the block layer because applications expect sector-oriented access and filesystem caching. A CAN controller that exchanges addressed frames belongs in the networking stack when exposed as a network interface.

4.2. register_chrdev() compared with misc_register()

Topic	register_chrdev() approach	misc device approach
Numbering	Driver chooses and manages a major number and its device range.	Uses shared major 10; driver selects or requests a minor number.
Node setup	A minimal exercise may use mknod to create /dev/asgn1.	Often created automatically by devtmpfs/udev after registration.
Boilerplate	More explicit number and node management.	Compact structure and a small registration path.
Advantages	Useful when the driver needs explicit device-number control or several minors.	Convenient for small, single-purpose devices.
Trade-off	Manual numbering and node creation can introduce setup errors.	The shared misc namespace is intended for relatively simple devices.

The assignment uses register_chrdev() for the ioctl character driver because it exercises major-number ownership and manual node creation. The counter module uses misc_register() because it presents a compact, single-purpose interface.

4.3. Synchronization choice

A plain integer updated by multiple concurrent readers can lose increments when two CPUs load the same value, calculate the next value, and store it concurrently. atomic_t provides atomic read-modify-write operations for the counter. For the first driver, the buffer and several related statistics must remain consistent as one state, so a mutex is more appropriate than independent atomic variables. Atomicity of the counter alone does not automatically make a multi-step read-and-increment sequence one indivisible operation.

4.4. Why the user/kernel boundary matters

Kernel code executes with elevated privilege and must not assume that a pointer supplied by an application refers to valid accessible memory. Safe user-copy functions provide controlled fault handling. Input lengths must be bounded, output structures initialized, and return values checked. These rules are as important for a simulated driver as for a physical SoC peripheral driver.

4.5. ioctl MAGIC NUMBER AND COMMAND ENCODING

The magic value groups commands associated with a driver, while the command number identifies one operation within that group. On the common generic Linux ioctl encoding used in the assignment, a request is composed from direction, size, type (magic), and number fields.

Bits (generic layout)	Field	Meaning
31-30	Direction (_IOC_DIR)	Data flow: none, write to driver, read from driver, or both.
29-16	Size (_IOC_SIZE)	Size of the argument type encoded by _IOR/_IOW/_IOWR.
15-8	Type (_IOC_TYPE)	Driver or subsystem magic value.
7-0	Number (_IOC_NR)	Operation identifier within that type.

For ASGN1_GET_VERSION, _IOR(ASGN1_MAGIC, 4, char[64]) specifies a kernel-to-user copy, encodes a 64-byte argument, and uses command number 4. The precise bit layout is architecture/API dependent in detail, so a driver should use the standard macros rather than hard-code request integers.

A request such as 0x6b7f in the provided reference does not match any supported command. The handler therefore takes its default branch and returns -ENOTTY. The application sees this as an ioctl failure with errno set to ENOTTY.

5. THEORY QUESTIONS

5.1. How does module reference counting prevent unsafe unloads?

When a device file is opened and its file_operations structure declares .owner = THIS_MODULE, the kernel can hold a module reference for the active file. The reference is released when the file is closed. If a process still has the device open, rmmod normally refuses to remove the module because its reference count is non-zero. The safe response is to close the open file descriptors and retry the unload. Forced unload is not a substitute for correct lifetime management because code or data may still be referenced after removal.

5.2. Why use copy_to_user() instead of memcpy()?

memcpy() is intended for ordinary memory regions and does not provide the user-pointer fault handling expected at the kernel boundary. A user address can be unmapped, read-only, or otherwise inaccessible. copy_to_user() and copy_from_user() use the kernel's user-memory access mechanisms and report how many bytes were not copied. The driver should check that return value and normally report -EFAULT when the transfer is incomplete. The helpers do not replace length validation or initialization of output data.

5.3. When should raw flash use MTD rather than the block layer?

Raw NAND or NOR flash has erase-block constraints, finite erase endurance, and may contain bad blocks or require error-correcting codes. The Memory Technology Device (MTD) subsystem exposes flash-oriented operations such as reading, writing, erasing, and handling bad blocks. Filesystems or layers such as UBI/UBIFS can build wear management and bad-block handling above it. eMMC and SD cards generally include a flash translation layer in their controllers and are presented to Linux as block devices, so applications access them through the block layer.

5.4. Why must read() handle *ppos?

The VFS passes the current file offset through ppos. After a positive read result, the offset advances and the caller may issue another read. For a counter interface that returns one formatted value per open, the first call can return the string and update the offset; the next call should return 0 to signal end-of-file. Without this check, programs such as cat could keep reading indefinitely because every invocation would return more digits. A new open normally starts with a fresh file offset.

6. EXPERIMENTAL EVIDENCE MATRIX

This matrix maps each screenshot to the behavior it verifies. Keep raw terminal output and kernel logs with the source code; filenames below are suggested labels.

Evidence ID	Suggested artifact	What it should verify
FIGURE 1	architecture_diagram.png	User-space tools, system-call boundary, device/proc interfaces, both driver modules, and the QEMU target.
EVIDENCE 01	task1_driver_logs_summary.png	Task 1 driver activity and the captured final result: 4 PASS, 0 FAIL.
EVIDENCE 02	task1_modes_version_stats.png	Echo/Kernel mode operation, GET_VERSION, and GET_STATS output.
EVIDENCE 03	task1_reset_invalid_ioctl.png	RESET_BUFFER statistics and rejection of an unsupported ioctl request.
EVIDENCE 04	task2_counter_test.png	Initial/final procfs values, reads 0/1/2, set/reset behavior, invalid input rejection, and 7 PASS, 0 FAIL.
LOGS	task1_test_output.txt; task2_test_output.txt; dmesg_output.txt	Raw terminal output and kernel logs retained with the source code.

7. CONCLUSION AND ENGINEERING EVALUATION

The assignment illustrates two patterns for exposing kernel functionality to user space. The character driver demonstrates a control interface built with ioctl commands, safe user-memory transfer, mode handling, shared-buffer protection, and explicit error codes. The counter driver demonstrates misc-device registration, procfs diagnostics, parsing of user input, atomic state updates, and correct end-of-file behavior.

For IC design and embedded engineering, a peripheral or hardware block needs more than RTL or register definitions. Integration also requires a safe kernel interface, synchronization, diagnostics, and repeatable tests.

REFERENCES

[1] Assignment_1.pdf, user-provided reference report, Embedded Linux Systems / Assignment 1, FPT University, Ho Chi Minh City Campus, 2026.

[2] The Linux Kernel Documentation, “ioctl based interfaces,” https://docs.kernel.org/driver-api/ioctl.html (accessed 21 October 2026).

[3] The Linux Kernel Documentation, “Miscellaneous Devices,” https://docs.kernel.org/driver-api/misc_devices.html (accessed 21 October 2026).

[4] Linux man-pages project, read(2), write(2), ioctl(2), and errno(3), for user-space system-call semantics.